Organización de Lenguajes y Compiladores 1

Proyecto 2

Primer Semestre 2026, Centro Universitario de Occidente

"Id y Enseñad a Todos"

Objetivos generales

- Aplicar conocimientos de análisis léxico y sintáctico.
- Aplicar traducción dirigida por la sintaxis.
- Explorar herramientas modernas para el desarrollo web
- Construir aplicaciones funcionales

Objectivos específicos

- Combinar la funcionalidad de la herramienta Jison haciendo uso del entorno de ejecución NodeJs en aplicaciones reales.
- Utilizar una tabla de símbolos primitiva.
- Aplicar los conocimientos adquiridos en el curso para la programación de aplicaciones funcionales

Descripción de la actividad

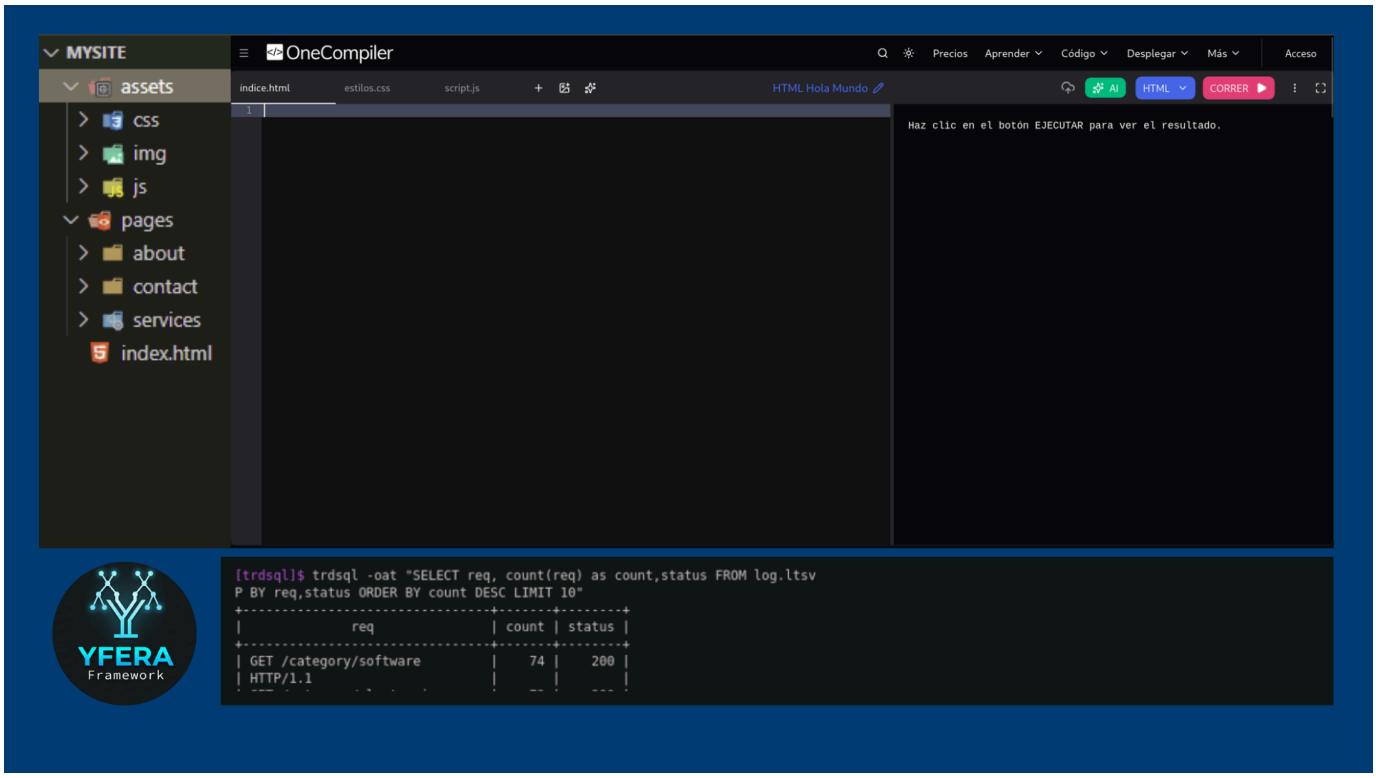
En la actualidad el uso de frameworks es fundamental para el desarrollo web y debido a su buen desempeño en el proyecto 1, la empresa YFERA le contrató para la programación de un framework personalizado, que le permita a los usuarios construir páginas web.

Descripción de la Aplicación

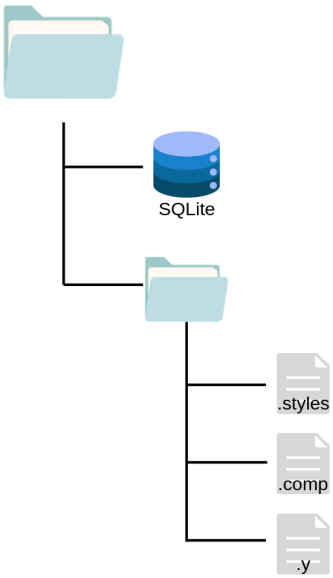
Se deberá de programar una aplicación web para que los usuarios puedan usarlo como IDE, esto con el objetivo de que ellos puedan construir sus propias aplicaciones web y que a futuro puedan ponerlas en producción.
El framework que se va a programar se llamará YFERA y los principios con los que está pensado son que sea fácil de aprender, que sea tipado y que a su vez sea potente, capaz de crear diferentes vistas.

Adicionalmente se pide que la aplicación sea amigable con el usuario, contando con ciertas funcionalidades que le permitan ser lo más eficiente posible. Estas características son:

- Poder exportar el árbol de trabajo
- Crear archivos y carpetas
- Abrir proyectos
- Reconocer comentarios en el código.
- Si existiera algún error léxico o sintáctico en la ejecución de alguna instrucción se deberá mostrar de la manera más informativa y amigable posible.
- Tener un selector de colores personalizado.
- Colorear el código que introduzca el usuario para que le sea más fácil la visualización del código.
- Indentar el código automáticamente.



Referencia para la interfaz



Referencia para un árbol de trabajo

Definición de los Lenguajes

Definición del lenguaje para estilos

Este lenguaje servirá para definir estilos globales que luego los componentes podrán utilizar. Este lenguaje es **case sensitive**, por lo que distinguirá entre minúsculas y mayúsculas. Este será muy similar a CSS con algunas modificaciones, tiene extensión .styles

Definición de estilos

Se podrá definir un estilo de la siguiente manera:

```
mi-clase {
    height = number;  /* alto por defecto*/
    width = number;  /* ancho por defecto */
    min-width / max-width / min-height / max-height = number ;  /*limita el grid*/

    background color = lightgray ; /* Color de fondo */
    color = color ; /* color de texto */

    text align = direction ; /* CENTER, RIGHT, LEFT */
    text size = 10 ;
    text font = font ; /* HELVETICA, SANS, SANS SERIF, MONO, CURSIVE */

    padding = 10 ; /* Espacio interno general */
    padding left = 10 ;
    padding top = 10 ;
    padding right = 10;
    padding bottom = 10;

    margin = 10 ; /* Espacio externo general*/
    margin left = 10 ;
    margin top = 10 ;
    margin right = 10 ;
    margin bottom = 10

    border radius = 10 ; /* redondear esquinas */
```

```
    border style = STYLE ; /* estilos permitidos DOTTED, LINE, DOUBLE */
    border width = 10 ;
    border color = color ;
    border = 2 solid red; /*abreviatura para el width style y color */

    /*
        es posible combinar las propiedades de los borderes con las 4 direcciones,
        por ejemplo:
    */
    border top style = solid;
    border right = 2 solid red;
}
```

La unidad de medida son px, excepto cuando se especifican porcentaje, por ejemplo 80%

Todas las propiedades de estilos son opcionales y pueden estar en cualquier orden. Solo se pueden aplicar esas propiedades.

Las propiedades se leen de arriba para abajo, es decir que se aplica la que está más abajo y se pueden sobreescribir entre sí, ejemplo:

```
mi-estilo {
    border color = red ;
    border right = 2 solid blue;
    /*el borde del lado derecho será azul pero todos los demás serán rojos*/
}
```

Adicionalmente los estilos se pueden basar en otros, por ejemplo:

```
super-estilo {
    background color = lightgray ;
}

mi-estilo extends super-estilo{
    color = blue ; /*el componente que lo utilice tendrá fondo gris y letras azules*/
}
```

Por último para que la generación de estilos sea más rápida, se permitirá un bucle for que no se puede anidar y que solo puede usar una variable entera que se define a la vez que se usa el bucle for, se permitirán expresiones únicamente numéricas dentro del for:

```
@for $i from 1 through 4 { /*de 1 hasta 4 incluyendolo*/
     /*
        el nombre de la variable se concatena y al final se generarían
        4 estilos: my-font-1, my-font-2, my-font-3, my-font-4
     */
    my-font-$i {
        text size = $i * 10;
    }
}

@for $i from 1 to 4 { /*de 1 hasta 4 excluyendolo*/
    my-font-$i {
        text size = $i * 10;
    }
}
```

Definición del lenguaje para componentes

Este lenguaje servirá para definir componentes; un componente es un fragmento de código que servirá para generar una vista. Tiene extensión .comp

Un componente se define de la siguiente manera:

```
component(){
    /* elementos */
}
```

Adicionalmente un componente puede requerir n parámetros de cualquier tipo que se le pueden pasar, por ejemplo:

```
component(int myNumber, string username, function myFunction){
    /* elementps */
}
```

Los nombres de los componentes no se pueden repetir.

Un componente puede tener diferentes elementos, los cuales se especifican a continuación:

Secciones

```
[
    /*esto es una sección*/
    [
        /* las secciones se pueden anidar*/
    ]
]

<mi-estilo, otro-estilo>[
    /* esto es una sección con estilos */
    <estilo-propio>[
        /*
            las secciones internas heredan los estilos de sus padres,
```

```
            aunque pueden tener sus propios estilos
        */
    ]
]
```

Tablas

```
[[
    [[
        [[  /* elemento 11 */ ]]
        [[  /* elemento 12 */ ]]
    ]]
    [[
        [[  /* elemento 21 */ ]]
        [[  /* elemento 22 */ ]]
     ]]
]]


<mi-estilo>[[
      /* elementos de las tablas aqui*/
]]
```

Texto

```
T<estilo-opcional, otro-estilo-opcional>("Aqui esta mi texto")
T("
     Dentro de TODOS textos debe ser posible usar variables
     que se le pasen al componente,
     por ejemplo la siguiente $myUsername

     Otra opción es realizar operaciones dentro de un texto
     Por ejemplo, la suma es `$total + $subtotal + 0.5`

     Tambien deben soportar múltiples lineas.
")
```

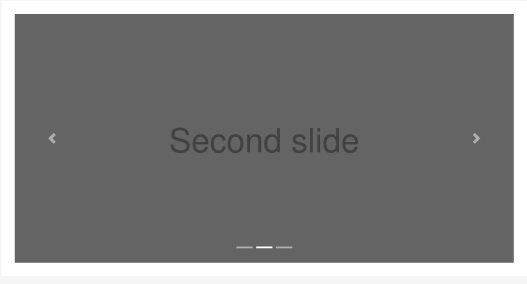
Imágenes

Las imágenes se pueden insertar individualmente:

```
IMG<estilo-opcional>("url1")
```

O en conjunto para crear un carrusel, es posible usar variables de tipo string para especificar urls

```
IMG<estilo-opcional>("url1", "url2", $url_3, $urls[1], "url_n")
```



Formularios

```
updatePkmComponent(
    function functionUpdatePokemon, string currentName, int $currentPP
){
    FORM<estilo-opcional>{
        /*
             dentro del cuerpo del formulario pueden ir los demás elementos,
             no sólo los inputs
             Los textos deben soportar variables con $var
             y también operaciones con `$var +1`
        */
        INPUT_TEXT<estilo-opcional>{
             id: "name",
             label: "Ingresa un nombre nuevo",
             value: "$currentName"
        )
        INPUT_NUMBER<estilo-opcional>(
            id: "pp",
            label : "Ingresa un pp",
            value : $currentPP
        )
```

```
        /* el input tipo booleano representa un checkbox */

        INPUT_BOOL<estilo-opcional>(
            id : "valid",
            label: "Es válido?",
            value: true
        )
    } SUBMIT<estilo-opcional> {
         /*
             esto es opcional, crea un botón al final para enviar el formulario,
             Dentro de esta sección no pueden haber otros elementos
         */

         label: "enviar formulario"
         /*se pueden pasarle los valores de los inputs del usuario usando el id*/
         function: $functionUpdatePokemon(@valid, @name)
    }
}
```

Lógica

Dentro de un componente se pueden tener ciclos, condicionales y casos para facilitar la visualización de información dinámica, sin embargo serán sencillos, pues no se recomienda que en la vista haya lógica más allá de la que sirve para mostrar la información. Todos estos elementos lógicos se deben poder anidar.

Ciclo For

```
/* ciclo for sencillo, para tener  */
for each ($myArrayVar : $currentAccess) {
     T("El actual elemento es $currentAccess")
}

/* ciclo for complejo */
for ($myArrayVar : $currentAccess, $anotherVar : $secondaryElement) track $index {
    T("
         El actual elemento es $currentAccess
         El elemento complementario es $secondaryElement
         y el índice actual de los elementos dentro del array es es $index
```

```
        para mejor visualizacion del usuario el indice es `$index + 1`
      ")
} empty {
     T("El arreglo está vacío")
}
```

Condicionales

Solo acepta que el resultado de la condición sea tipo booleano.

```
if( $myVar == "Hola" ) {
    /* elementos */
} else ( $myVar == "Adios" && $boolVar ){
    /* elementos */
} else {
    /* elementos */
}
```

Switch

Acepta variables tipo entero y string, se asume que dado un caso no se ejecutara el código de otro caso.

```
Switch( $myVar ) { /* Esto es valido: $arrayVar[0] */
    case "something"{
        /* elementos */
    },
    case "something else"{
        /* elementos */
    },
    default { /* opcional */
       /* elementos */
    }
}
```

Definición de lenguaje para interactuar con la DB

En el proyecto se debe permitir interactuar con una consola de sql, que internamente se comunicara con una base de datos de sqlite. Este lenguaje es bastante sencillo, limitándose a que pueda realizarse CRUD en tablas simples sin el uso de JOINS.
Para la creación de una tabla se debe dejar por defecto que tenga una columna llamada id que será la llave primaria.

```
/*
    Crear una tabla,
    los tipos que se manejan son los mismos que los del lenguaje principal
*/
TABLE table_name COLUMNS column_name=type, column2_name=type;

/*
    Seleccionar todas las columnas de una tabla particular,
    esto devuelve un array del tipo de la columna
*/
table_name.column_name;

/*
    Crea un registro
*/
table_name[column_name="value", column2_name=0.0]

/*
    Actualiza un registro,
    la condición es obligatoria y siempre compara con el id
*/
table_name[column_name="new value", column2_name=10.10] IN 1;

/*
    Elimina  un registro de una tabla,
    la condición es obligatoria y siempre compara con el id
*/
table_name DELETE 1;
```

Definición del lenguaje principal

Este lenguaje es el encargado de orquestar la lógica primaria con los componentes y estilos. Su extensión es .y

La estructura de este es el siguiente:

```
/*
     imports que se cargan al principio, los paths siempre deben ser relativos
     si no se encuentra un import se debe mostrar un error
 */
import "./carpeta1/misComponentes.comp ";
import "./carpeta2/misEstilos.styles ";

/*
     definicion de variables
*/

int variableEntera = 2;
float variableConDecimales = 10.0;
string variableDeTexto = "Holaaa";
boolean variableBooleana = True; /* or False */
char variableDeUnSoloCaracter = 'a' /* o cualquier código ASCII*/

# los arreglos pueden ser de cualquier tipo,
# limitados a que solo pueden ser de una dimensión
int[] arregloDeEnteros = [3]; /* arreglo de 3 posiciones */
string[] pokemons = {"Bulbasaur", "Togepy", "Eevee" }; /*  arreglo de 3 dimensiones
ya inicializado*/

# es posible consultar la base de datos y asignarle un valor a la variable
float[] myTeam = execute `my_sql_table_for_pkms.name`

/*
    las funciones en este lenguaje serán simples,
    limitándose únicamente a la interacción con la base de datos de datos,
    solo lo que se especifica en el cuerpo de las siguientes funciones
       son las instrucciones válidas
```

```
*/


function updatePokemon(int pp, int atack, int id, string goTo){
    # permite ejecutar instrucciones para la base de datos
    # si ocurre un error se debe dejar por defecto que se muestre
      una alerta en el navegador y detener la ejecución de lo siguiente
    execute `my_pokemon_team[power_points=$pp, atack=$atack] IN $id`;
    # recarga la ejecución de un archivo, en este caso de una variable
    load goTo;
}

function goTo(){
    # recarga la ejecución de todo el archivo principal
    # no se pueden usar funciones dentro de funciones
    load "./main.y";
}


/*
     Esta es la función principal que se encarga de cargar componentes
*/
main {
    @header();    # los componentes se pueden invocar
    while(variableEntera < 10){
        @lineForStyle();
        variableEntera = variableEntera + 1;
    }

    for(i = 0; i < 10; i = i ++){
        @Card(pokemons_name[i], pokemons_id[i], updatePokemon);
    }

    if( variableBooleana ) {
        @cardForProfOak();
    } else {
        @cardForAsh(pokemons_name[i]);
    }
```

```
    @footer();
}
```

Bloques de código

Son bloques de código que incluyen lógica. Todos los bloques de código se pueden anidar.

Condicionales

If

```
if ( 10 > 10 ) {

} else if ( 10 < 50 ) {

} else  {

}
```

Las condiciones deben ser de tipo booleano.

Switch

```
switch (myVar) {
     case "a":

         break;
     case "b":
     case "c":

        break;
     default:
        break;
}
```

Ciclos

Se debe permitir el uso de break y de continue para los ciclos.

Ciclos while y do-while

```
while ( 10 > 10 ) {

}
```

```
do {

} while ( 10 < 10 || 10 < 10 )
```

Ciclo For

```
for (i = 0 ; i <= 10 ; i = i + 1 ) {

}
```

Definición de expresiones

La siguiente lista de expresiones aplica para todos los lenguajes en el proyecto, cuando se requieran. Se consideran expresiones, aquellas que ejecutarán alguna operación aritmética. En las operaciones aritméticas se deben incluir el uso de variables, según su tipo y también dependiendo del lenguaje específico.

Operadores aritméticos

Dentro del lenguaje se podrán utilizar, valores numéricos con o sin punto decimal con los cuales se pueden ejecutar operaciones aritméticas básicas las cuales se definen en la siguiente tabla.

Nota: Hay que tener en cuenta la operación unaria, es decir números con signos.

Símbolo	Descripción	Precedencia (de menor a mayor)
+	Suma	1
-	Resta	1

*****	Multiplicación	2
/	División	2
%	Módulo	3
( )	Paréntesis	4
-	Unario	4

Operadores de comparación

Se usan para comparar diferentes valores

Símbolo	Descripción	Precedencia (de menor a mayor)
>	Mayor que	0
>=	Mayor o igual que	0
<	Menor que	0
<=	Menor o igual que	0
==	Igualdad	0
!=	Diferente	0

Nota: en el caso de los strings, se hace la comparación lexicográfica de los mismos.

Operadores lógicos

Los operadores lógicos son símbolos o palabras que conectan dos o más expresiones booleanas para formar una nueva expresión.

Símbolo	Descripción	Precedencia (de menor a mayor)
||	o	1
&&	y	1
!	Negación	2

Reporte de errores

Ya sea en una notificación que se pueda cerrar, en un apartado especial para errores u otro; se debe mostrar un reporte de errores para ayudar al usuario a corregir su código, este debe ser similar a lo siguiente:

Lexema	Línea	Columna	Tipo	Descripción
&	2	13	Léxico	Símbolo no existe en el lenguaje
*	3	1	Sintáctico	Se esperaba ‘graficar’ o ‘animar’

Otras funcionalidades

Gestión de archivos

Se debe poder crear un nuevo proyecto, abrir un árbol de trabajo y descargarlo. Así como descargar el código final para visualización (código final html).

Coloreado del código

Para mejorar la visualización del código, se colorearán ciertos elementos. Esto aplica únicamente para el lenguaje para la creación de formularios, no para el lenguaje de guardado, que nunca se mostrará en la aplicación web. Se tiene la siguiente tabla para referencia:

Elemento	Color
Operadores	Verde
Variables	Blanco
Strings literales	Naranja
Otros literales	Celeste
Palabras reservadas	Morado

llaves, corchetes, paréntesis	Azul
Otros	Blanco

Selector de colores

Para facilidad del usuario se le debe dar la opción de seleccionar el color que quiera para su componente e insertarlo en cualquier formato en el código, aplica para el lenguaje de estilos.

Indentado Automático

Se debe poder indentar todos los lenguajes para facilitar su lectura.

Importante

- Lenguajes de programación y Frameworks válidos: Angular, Nodejs, JavaScript, Typescript, HTML, CSS, Bootstrap.
- Usar la herramienta Jison para cualquier tipo de análisis léxico y sintáctico.
- Práctica obligatoria para tener derecho al siguiente proyecto.
- Las copias obtendrán nota de cero y se notificará a coordinación.
- Si se va a utilizar código de internet, entender la funcionalidad para que se tome como válido.

Entrega

La fecha de entrega es el día **07 de mayo del 2026 a las 14:00 horas.**
Los componentes a entregar en repositorio de github son:

- Código fuente (Proyecto completo con todas sus dependencias o módulos internos y externos)
- Manual técnico de la aplicación con un apartado del procedimiento para construir las gramáticas utilizadas y la explicación de cada una de ellas, así como el diagrama de clases.
- Manual de usuario (Intuitivo).

Calificación

Pendiente de definir.